Système de Gestion des Stages — STEG

Plateforme complète de gestion des stages au sein de la STEG (Société Tunisienne de l'Électricité et du Gaz). Elle permet de piloter l'ensemble du cycle de vie d'un stage : offres, candidatures, affectation des stagiaires, suivi des projets et des tâches, documents, évaluations, réunions et notifications — avec un contrôle d'accès par rôle et par propriété des ressources.

---

1. Vue d'ensemble

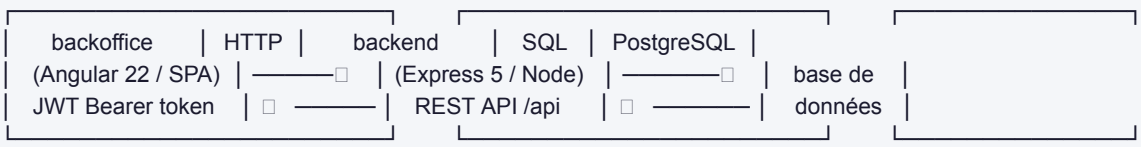
Élément	Détail
---	---
Type	Application web full-stack (gestion de stages)
Backend	REST API — Node.js / Express 5, PostgreSQL
Frontend	Single-page application — Angular 22 (template Berry)
Authentification	JWT (Bearer token) + bcrypt
Tests backend	Jest + Supertest — **71 tests / 3 suites**
Tests frontend	Karma / Jasmine — **83 tests**
Serveur API	http://localhost:3000/api
Application web	http://localhost:4200

Le projet est organisé en deux modules indépendants :

```
stage26/
├── backend/      # API REST (Node.js, Express, PostgreSQL)
├── backoffice/   # Application Angular (interface d'administration)
└── README.md     # Ce document
```

---

2. Architecture



2.1 Backend — architecture modulaire (en couches)

```
backend/
├── server.js           # Point d'entrée, configuration Express
├── routes/             # Définition des routes (1 fichier par ressource)
├── controllers/        # Couche HTTP : lecture de la requête, réponse
├── services/           # Logique métier
├── repositories/       # Accès aux données (SQL PostgreSQL)
├── validators/         # Règles de validation (express-validator)
├── middlewares/        # Auth JWT, rôles, autorisation, sécurité, rate-limit
├── utils/              # Helpers (scope, pagination, réponse, erreurs…)
├── database/           # schema.sql, seed.js, migrations
├── config/             # Connexion PostgreSQL, config uploads
└── __tests__/          # Tests d'intégration (Jest)
```

Le flux d'une requête : **route → middleware (auth / rôle / autorisation) → controller → service → repository → PostgreSQL**

2.2 Frontend (backoffice)

Application Angular 22 basée sur le template admin **Berry** (Cod

```
backoffice/src/app/
├── core/         # Data, guards, interceptors, layout, shared
├── theme/        # Thème, services partagés, directives
├── demo/         # Pages de démonstration du template
└── app-routing.module.ts
```

Un interceptor HTTP attache automatiquement le token JWT aux requêtes ; les guards de route protègent l'accès aux écrans selon le rôle connecté.

---

3. Fonctionnalités

3.1 Authentification

- Connexion avec email / mot de passe (`POST /api/auth/login`), ém
- Récupération du profil courant (`GET /api/auth/me`).
- Mots de passe hachés avec **bcrypt** (12 rounds).
- **Limitation de débit** : `authLimiter` (10 connexions / 15 min / IP) et

3.2 Domaines métier

Domaine	Description
---	---
Offres de stage	Publication, ouverture/fermeture, consultation par les stagiaires
Candidatures	Dépôt d'une candidature à une offre, traitement par la RH (EN_ATTENTE / ACCEPTEE / REFUSEE)
Stagiaires	Enregistrement des profils (école, spécialité, niveau, compétences)
Encadrants	Gestion des superviseurs de stage
Départements	Organisation des services d'accueil, chef de département
Stages	Affectation stagiaire ↔ encadrant ↔ département, statuts (EN_COURS, TERMINE, SUSPENDU, ANNULE)
Projets & Tâches	Suivi des travaux confiés au stagiaire, responsables et exécutants
Documents	Upload / téléchargement / aperçu (CV, convention, assurance, rapport, attestation)
Évaluations	Notation par l'encadrant (note et appréciation)
Réunions	Planification des réunions de suivi
Rapports	Suivi des rapports de stage
Notifications	Fil de notifications par utilisateur (lu / non lu)
PDF	Génération automatique de la convention, de l'attestation et de l'évaluation (pdfkit)
Tableau de bord	Statistiques agrégées par rôle (stagiaires, offres, départements)
Recherche	Recherche globale multi-entités, scopée par rôle

4. Rôles et contrôle d'accès

Cinq rôles sont gérés (enum PostgreSQL role_utilisateur) : ADMIN, RH, CHEF_DEPARTEMENT, ENCADRANT, STAGIAIRE.

4.1 Matrice d'accès par rôle

Ressource	ADMIN / RH	CHEF_DEPARTEMENT	ENCADRANT	STAGIAIRE
---	---	---	---	---
Toutes ressources	□ Accès total	—	—	—
Stagiaires	□ Tous	□ Son département	□ Ses stagiaires	□ (sauf profil)
Stages	□ Tous	□ Son département	□ Les siens	□ Le sien
Candidatures	□ Toutes	□ Offres de son département	□ Offres de son département	□ Les siennes
Offres	□ Toutes	□ Son département	□ Son département	□ Offres ouvertes uniquement
Tâches	□ Toutes	□ Projets de son département	□ Les siennes / projets	□ Les siennes
Réunions	□ Toutes	□ Son département	□ Les siennes	□ Les siennes
Évaluations	□ Toutes	□ Son département	□ Les siennes	□ La sienne
Documents	□ Tous	□ Son département	□ Ses stages	□ Les siens
Encadrants	□ Tous	□ Son département	—	—
Départements	□ Tous	□ Le sien	—	—

4.2 Couche d'autorisation par propriété

Au-delà des rôles, l'accès à un objet précis est vérifié par une couche d'autorisation dédiée qui garantit que l'utilisateur n'accède qu'aux ressources lui appartenant (ou à son périmètre), même s'il possède le bon rôle :

- backend/utils/scope.util.js — scopeFilter(scope, resourceType) génère le WHERE SQL adapté à chaque rôle (filtrage des listes).
- backend/services/authorization.service.js — resolveScope(user) (périmètre de l'utilisateur) et loadAndAssert(resourceType, id, user, scope) (vérification objet par objet).
- backend/middlewares/authorization.middleware.js — middlewares réutilisables withScope, authorizeResource(...), authorizeStage(...), assertValidUuid(...).

Comportement : ressource absente ou identifiant non-UUID → 404 ; ressource existante mais hors périmètre → 403. La réponse d'erreur est au format JSON standardisé.

Les 9 listes (/api/stagiaires, /api/stages, /api/candidatures, /api/reunions, /api/evaluations, /api/taches, /api/offres, /api/departements, /api/encadrants) sont filtrées par périmètre, et toutes les opérations de modification (PUT/DELETE) sur un objet sont protégées par authorizeResource.

---

5. Base de données

5.1 Tables (PostgreSQL)

```
utilisateur ──┬── administrateur
              ├── rh
              ├── chef_departement
              ├── stagiaire
              └── encadrant ── departement

departement ──┬── chef_departement
              ├── offre_stage ── candidature ── stagiaire
              ├── stage ──┬── stagiaire
              │           ├── encadrant
              │           ├── projet ── tache
              │           ├── document
              │           ├── rapport
              │           └── evaluation ── encadrant
              └── reunion

permission / utilisateur_permission   (droits fins, extensible)
notification                          (par utilisateur)
```

5.2 Enums

- role_utilisateur : ADMIN, STAGIAIRE, ENCADRANT, CHEF_DEPARTEMENT, RH
- statut_candidature : EN_ATTENTE, ACCEPTEE, REFUSEE
- statut_stage : EN_COURS, TERMINE, SUSPENDU, ANNULE
- statut_offre : OUVERTE, FERMEE, EXPIREE, POURVUE
- type_document : CV, CONVENTION, ASSURANCE, RAPPORT, ATTESTATION

Le script DDL se trouve dans backend/database/schema.sql, les données de démonstration dans backend/database/seed.js.

---

6. API REST — Résumé des routes

Toutes les routes sont préfixées par /api et protégées par authentification JWT (sauf /api/auth/login).

Méthode & chemin	Description
---	---
POST /api/auth/login	Connexion → token JWT
GET /api/auth/me	Profil de l'utilisateur connecté
GET/POST /api/stagiaires · GET/PUT/DELETE /api/stagiaires/:id	Gestion des stagiaires
GET/POST /api/encadrants · PUT/DELETE /api/encadrants/:id	Gestion des encadrants
GET/POST /api/departements · PUT/DELETE /api/departements/:id	Gestion des départements
GET/POST /api/offres · PUT/DELETE /api/offres/:id	Offres de stage
GET/POST /api/candidatures · PUT/DELETE /api/candidatures/:id	Candidatures
GET/PUT/DELETE /api/stages · GET/PUT/DELETE /api/stages/:id	Stages
GET/POST /api/taches · PUT/DELETE /api/taches/:id · GET /api/taches/mes-taches	Tâches
GET/POST /api/reunions · DELETE /api/reunions/:id	Réunions
GET/POST /api/evaluations · PUT/DELETE /api/evaluations/:id	Évaluations
POST /api/documents/stages/:stageId/documents · GET .../documents	Upload / liste des documents
GET /api/documents/:id/download · GET /api/documents/:id/preview · DELETE /api/documents/:id	Documents
GET /api/stages/:stageId/pdf/convention · /pdf/attestation · /pdf/evaluation	Génération PDF
GET /api/search	Recherche globale
GET /api/dashboard/stats	Statistiques du tableau de bord
GET/PUT/DELETE /api/notifications	Notifications

Le format de réponse standard est { success, message?, data? } ; les listes paginées renvoient { data, total, page, pages }. Les erreurs sont renvoyées au format JSON standardisé { success: false, message, ... } (400 validation, 401 non authentifié, 403 non autorisé, 404 introuvable).

---

7. Sécurité

- Authentification JWT (24 h), en-têtes protégés par Helmet + en-têtes personnalisés (security.middleware.js).
- **bcrypt** (12 rounds) pour les mots de passe ; aucun mot de passe en clair.
- Contrôle d'accès : middleware de rôle (requireRole) + autorisation par propriété (withScope / authorizeResource / authorizeStage) — renvoie 403 hors périmètre, 404 si la ressource n'existe pas.
- **Limitation de débit** (express-rate-limit) sur /api/auth/login et l'API g
- **Validation des entrées** systématique (express-validator) avant toute opération.
- **Upload** : multer avec vérification de type et taille, nettoyage des fichiers orphelins en cas de rejet.
- **CORS** restreint à http://localhost:4200 (configurable via CORS_ORIG
- Énumérations stockées en base : les valeurs de rôle/statut ne peuvent pas être inventées.

---

8. Installation et lancement

Prérequis

- **Node.js** ≥ 18

PostgreSQL • ≥ 14
npm

8.1 Base de données

```
createdb steg_stages          # ou le nom configuré dans .env
psql -d steg_stages -f backend/database/schema.sql
node backend/database/migrate.js
node backend/database/seed.js
```

- Les scripts `migrate.js` et `seed.js` doivent être exécutés depuis le dossier `backend/` (ils utilisent `dotenv`).

8.2 Backend (API REST)

```
cd backend
cp .env.example .env   # si absent, puis renseigner DB_* et JWT_SECRET
npm install
node server.js         # serveur sur http://localhost:3000
```

Fichier .env attendu :

```
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_NAME=steg_stages
DB_USER=<utilisateur>
DB_PASSWORD=<mot de passe>
JWT_SECRET=<clé secrète>
JWT_EXPIRES_IN=24h
CORS_ORIGIN=http://localhost:4200
```

8.3 Frontend (backoffice)

```
cd backoffice
npm install
npm start              # application sur http://localhost:4200
```

---

9. Utilisateurs de démonstration (seed)

Rôle	Email	Mot de passe
---	---	---
ADMIN	admin@steg.tn	admin123
RH	salma.bouazizi@steg.tn	rh123
CHEF_DEPARTEMENT	nabil.khelifi@steg.tn	chef123
ENCADRANT	ahmed.bensalah@steg.tn	encadrant123
ENCADRANT	rachid.mansour@steg.tn	encadrant123
STAGIAIRE	fatma.trabelsi@steg.tn	stagiaire123
STAGIAIRE	mohamed.bouchnak@steg.tn	stagiaire123

Départements seedés : **Électromécanique** , **Finance** , **Informatique**

---

10. Tests

Backend — tests d'intégration (Jest)

```
cd backend
npm test
```

71 tests / 3 suites :

Suite	Objet	Nombre de tests
---	---	---
api.integration.spec.js	Fonctionnement global de l'API	32
authorization.integration.spec.js	Contrôle d'accès par rôle et par propriété	39
document-upload.integration.spec.js	Upload / validation des documents	11

- Les tests d'intégration s'exécutent **contre le serveur live** (`http://localhost:3000/api`). Le serveur doit être **redémarré avant chaque exécution** (`kill $(lsof -ti :3000); nohup node server.js > server.log 2>&1 &`), car la limite de connexions (`authLimiter` : 10 / 15 min / IP) est **en mémoire** et se réinitialise au redémarrage.

Frontend — tests unitaires (Jest/Karma)

```
cd backoffice
npx ng test --watch=false
```

83 tests — tous au vert.

---

11. Scripts utiles

Action	Commande
---	---
Démarrer l'API	cd backend && node server.js
Tester le backend	cd backend && npm test
Tester le backend (watch)	cd backend && npm run test:watch
Démarrer le frontend	cd backoffice && npm start
Tester le frontend	cd backoffice && npx ng test --watch=false
Rejouer le schéma DB	psql -d steg_stages -f backend/database/schema.sql
Rejouer les données	cd backend && node database/seed.js

---

12. Technologies

Couche	Technologies
Backend	Node.js, Express 5, PostgreSQL (node-postgres), jsonwebtoken, bcrypt, express-validator, multer, pdfkit, helmet, express-rate-limit, cors, dotenv
Frontend	Angular 22, Bootstrap 5, ng-bootstrap, ApexCharts, ng-apexcharts, TypeScript
Tests	Jest, Supertest (backend) · Karma, Jasmine (frontend)